

Games AI and Simulation

William Hammond

Temple Relic Defence

1. General Description of the Game

Temple Relic Defence is a 2D MonoGame prototype where the player controls a guardian protecting a relic in the centre of a temple. The player must survive for a set round duration while stopping thieves from reaching the relic, stealing it, and escaping through the exits. The level is built from rectangular walls that create corridors and pathways around the relic chamber. This means enemies cannot simply move in a straight line toward their target, as they must navigate around obstacles.

The player can move around the room using keyboard input and shoot projectiles toward the mouse. Projectiles can stun thieves, interrupt a theft attempt, and knock physics boulders around the room. Rolling boulders can then crush thieves, creating an environmental weapon rather than relying only on direct shooting. The relic also moves back toward its original home position when it has been dropped and is not being carried by a thief.

There are two main enemy types. Standard thieves prioritise the relic. They patrol, seek the relic, attempt to steal it, and escape if successful. A juggernaut thief behaves differently. Instead of targeting the relic, it targets the player and stuns them on contact. After stunning the player, the juggernaut respawns at the safest available spawn point furthest from the player to reduce the chance of stun-locking.

The project demonstrates basic simulation through movement, velocity, mass, collision handling, boulder knockback, and steering forces. It demonstrates basic AI through a finite state machine controlling the standard thieves. It also includes beyond-syllabus AI through grid-based A* pathfinding, allowing enemies to navigate around walls and corridors rather than becoming stuck on obstacles.

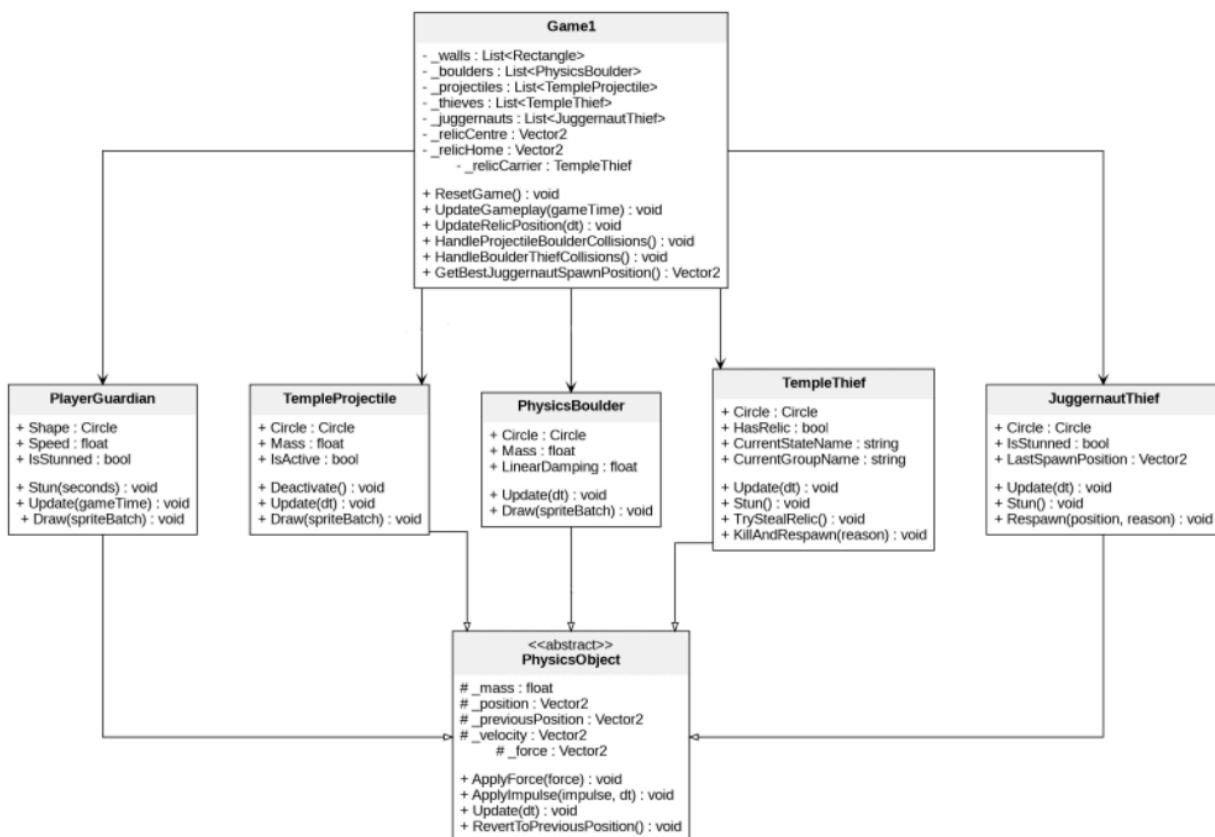
2. General Project Structure

The project is split between the main game project and a reusable game library. The reusable library contains general systems such as physics objects, shapes, steering behaviours, and finite state machine classes. The main game project uses these systems to implement the player, enemies, projectiles, boulders, relic logic, and level layout.

Important classes include:

- Game1.cs, which controls the main game loop, level setup, collision checks, spawning, relic behaviour, and screen states.
- PhysicsObject.cs, which provides reusable physics data such as position, velocity, force, and mass.
- PlayerGuardian.cs, which handles player movement, wall collision, drawing, and stun behaviour.
- TempleProjectile.cs, which represents the player's projectile.
- PhysicsBoulder.cs, which represents heavy movable boulders.
- TempleThief.cs, which controls normal thief movement, state machine behaviour, stealing, stunning, and pathfinding.
- JuggernautThief.cs, which controls the heavy player-hunting enemy.
- GridPathfinder.cs, which implements A* pathfinding through the level.
- ThiefStates.cs and ThiefTransitions.cs, which define the finite state machine states and transition rules.

Figure 1: Core Gameplay Class Diagram



3. Basic Simulation Using Newtonian Physics

The basic simulation in this project is built around the reusable `PhysicsObject` class in `MGGameLibrary/Abstract/PhysicsObject.cs`. This class stores mass, position, previous position, velocity, and force. Its `Update(float deltaTime)` method applies a simple Newtonian physics model where acceleration is calculated from force divided by mass:

```
Vector2 acceleration = _force / _mass;  
_velocity += acceleration * deltaTime;  
_position += _velocity * deltaTime;
```

This means the project uses the relationship between force, mass, acceleration, velocity, and position. The movement is updated over time using `deltaTime`, which keeps motion frame-rate independent.

The player, projectiles, boulders, thieves, and juggernaut all inherit from or use this physics foundation. For example, `TempleProjectile.cs` gives projectiles a small mass of 0.5f, while `PhysicsBoulder.cs` gives boulders a much larger mass of 12f. This difference matters during projectile-to-boulder collisions in `Game1.cs`, where the projectile impact is converted into boulder velocity using the mass ratio:

```
float kickSpeed = proj.Velocity.Length()  
    * (TempleProjectile.Mass / PhysicsBoulder.Mass)  
    * KickScale;  
boulder.Velocity += pushDir * kickSpeed;
```

This creates a simple but effective physical interaction. A light projectile does not simply teleport or instantly move the boulder. Instead, it applies a scaled knockback effect based on the projectile speed and relative mass. The boulder then continues moving with velocity and gradually slows down using damping in `PhysicsBoulder.cs`:

```
Velocity *= LinearDamping;
```

The boulders also interact with walls and other boulders. Wall collision is handled by checking a circular boulder against rectangular walls. If a collision is detected, the boulder reverts to its previous position and reflects velocity away from the wall. Boulder-to-boulder collisions are handled in `Game1.HandleBoulderBoulderCollisions()`, where overlapping boulders are separated and their velocity components along the collision normal are exchanged. This gives the boulders a more physical feel and allows them to act as moving hazards in the level.

The project also uses steering forces for AI movement. In `TempleThief.cs` and `JuggernautThief.cs`, enemies do not directly teleport to a target. Instead, they use a

SeekBehaviour combined with TruncatedSumSteeringBehaviour to calculate a steering force toward their current path target. This force is then converted into a movement velocity in ApplySteeringVelocity(). This connects the AI movement to the simulation layer, because the AI chooses a target while the steering system controls how the enemy moves toward it. Collision detection is another major part of the simulation. The project uses circular collision shapes for projectiles, boulders, thieves, the juggernaut, and the player. It also uses circle-to-rectangle checks for wall collisions. This is shown in methods such as PlayerGuardian.CircleIntersectsRectangle(), PhysicsBoulder.CircleIntersectsWall(), and Game1.CircleIntersectsRect(). These collision checks allow the game to have a more interesting level layout with walls, corridors, boulders, enemies, and projectile interactions.

Overall, the simulation system achieves the required basic physics by using mass, velocity, acceleration, force, collision response, wall collision, boulder damping, projectile movement, and AI steering.

4. Basic AI Technique: Finite State Machine

The main basic AI technique used in the project is a Finite State Machine. This is used to control the standard thief enemies. The generic finite state machine is implemented in MGGameLibrary/Graphs/FiniteStateMachine.cs. It uses states that implement IState and transitions that implement IStateTransition.

The finite state machine works by checking the outgoing transitions from the current state. If a transition condition returns true, the current state exits, the new state enters, and the new state is updated. This is shown in the Update(float seconds) method of FiniteStateMachine.cs.

The thief-specific states are implemented in ThiefStates.cs. These include:

- PatrolState, where the thief follows patrol waypoints.
- SeekRelicState, where the thief moves toward the relic.
- StealRelicState, where the thief stops and attempts to steal the relic after a short delay.
- EscapeState, where the thief carries the relic toward an exit.
- FleePlayerState, which is now only used as a fallback when the relic is not available.
- StunnedState, where the thief is temporarily disabled after being hit.

The transitions are implemented in ThiefTransitions.cs. Examples include:

- CanSeeRelicTransition
- CloseToRelicTransition
- HasRelicTransition
- HitByProjectileTransition
- StunFinishedTransition
- ReachedExitTransition

The actual thief finite state machine is built in `TempleThief.BuildFiniteStateMachine()`. This method creates all of the thief states, adds them to a graph, and connects them with transitions. For example, a thief can move from `Patrol` to `SeekRelic` if it can see the relic. It can move from `SeekRelic` to `StealRelic` when it is close enough to the relic. If the thief successfully steals the relic, it transitions to `Escape`. The thief uses sensor data to decide which transitions are valid. This data is updated in `TempleThief.UpdateSensors()`. The method checks conditions such as whether the relic is available, whether the thief is close to the relic, whether the player is nearby, whether the thief is stunned, and whether the thief has reached the exit. These values are stored in the `ThiefAgent` object and are then read by the transition classes.

A key design decision was to make thieves prioritise the relic over avoiding the player. Earlier versions allowed the player proximity to override the relic objective too often. The final version changes this so that standard thieves continue seeking or stealing the relic even if the player is nearby. The player must actively shoot or use boulders to stop the thief. This improves the challenge and makes the AI behaviour more purposeful.

5. Beyond-Syllabus Element: A* Pathfinding

The strongest beyond-syllabus AI feature in this project is the custom A* pathfinding system in `GridPathfinder.cs`. Basic seek steering was not enough once the level had corridors and walls. When enemies only moved directly toward the relic, they could become stuck against obstacles. To solve this, I implemented a grid-based pathfinder that allows thieves and juggernauts to find routes around walls.

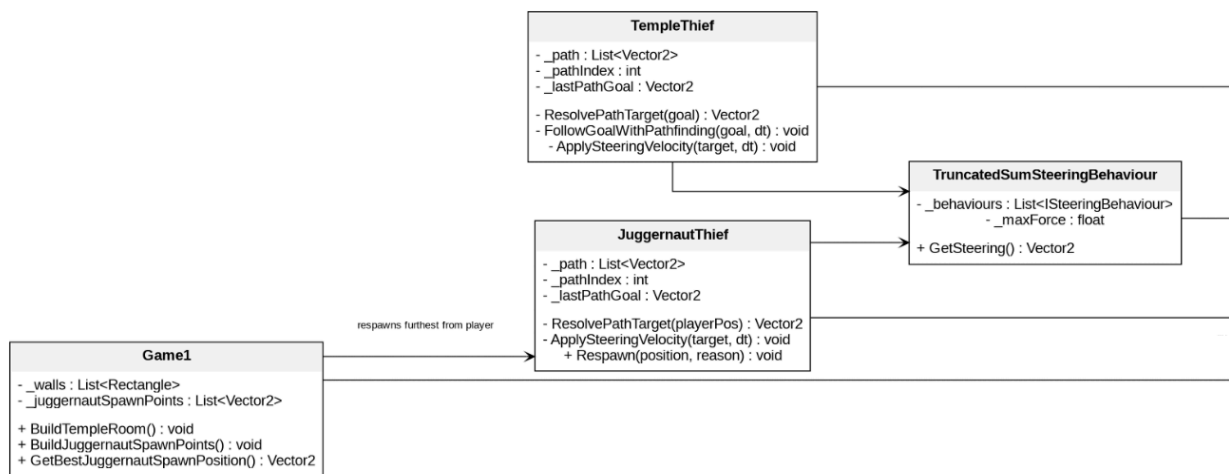
The pathfinder converts the game world into a grid. Each cell is marked as walkable or blocked. The blocked cells are generated from the level's wall rectangles. The walls are also inflated based on the enemy radius so that enemies do not path too close to corners or try to squeeze through gaps that are too small for their collision shape. The main method is `GridPathfinder.FindPath(Vector2 startWorld, Vector2 goalWorld)`. It converts the start and goal world positions into grid cells, finds the nearest walkable cells if either position is blocked, then searches for a path using A*. The heuristic uses distance between grid cells, and diagonal movement is supported. However, `GetNeighbours()` prevents diagonal corner-cutting by checking that both adjacent cardinal cells are walkable before allowing a diagonal move.

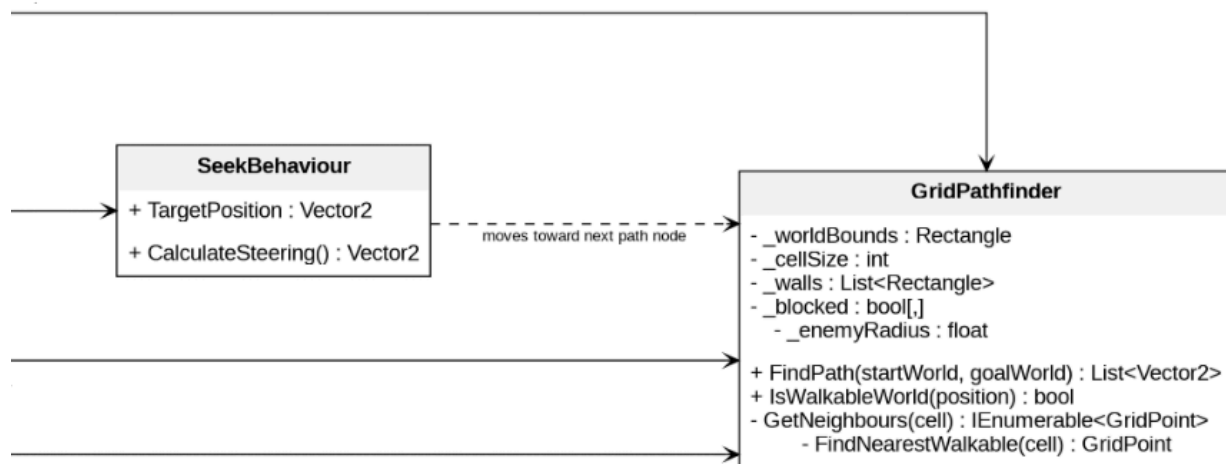
This pathfinder is then used by `TempleThief.ResolvePathTarget()` and `JuggernautThief.ResolvePathTarget()`. The finite state machine still decides the enemy's goal, such as the relic, exit, patrol point, or player. The pathfinder then decides how to reach that goal safely through the level. This separation is important because it keeps the decision-making system separate from the movement/navigation system. Extra logic was also added to make the pathfinding more reliable during gameplay. Enemies recalculate their path when the goal moves, when the enemy has moved far enough from the previous path start, or after a short refresh timer. They also skip path nodes when there is a clear line of sight to a later node, making

movement smoother. Stuck recovery was added so that if an enemy is not making enough progress, it forces a path recalculation.

This goes beyond a simple finite state machine because the AI is not only choosing states. It is also reasoning about the level layout and generating a route through a navigable space.

Figure 2: Pathfinding AI Navigation





6. Additional Beyond-Syllabus Element: Mass-Based Interactions

The project also goes beyond the basic requirements through the use of different masses and physics interactions between projectiles and boulders. The marking brief gives objects with different masses as an example of a possible beyond-syllabus simulation feature, and this project uses that idea directly.

The projectile has a mass of 0.5f, the standard thief uses a mass of 2f, the juggernaut uses a mass of 5f, and the boulder uses a mass of 12f. The most obvious gameplay effect is between projectiles and boulders. In `Game1.HandleProjectileBoulderCollisions()`, the projectile velocity and mass ratio affect how strongly the boulder is knocked back. This means the boulder acts like a heavy object rather than a lightweight object.

This also creates emergent gameplay. The player can use projectiles not only to hit thieves directly but also to knock boulders into enemies. In `Game1.HandleBoulderThiefCollisions()` and `Game1.HandleBoulderJuggernautCollisions()`, boulders only kill enemies if they are moving faster than `BoulderKillSpeed`. This prevents still boulders from being unfair instant-kill objects while still allowing fast rolling boulders to act as hazards. This improves the simulation because it gives physical objects different roles and behaviours. Projectiles are light and fast, boulders are heavy and slow, and enemies respond differently depending on whether they are stunned, crushed, or carrying the relic.

7. Juggernaut Enemy Extension

The juggernaut thief is an additional enemy type implemented in `JuggernautThief.cs`. Unlike the standard thieves, the juggernaut does not attempt to steal the relic. Its goal is to chase the player. This creates pressure on the player and prevents them from only camping near the relic. The juggernaut uses the same pathfinding idea as the normal thieves, but its target is the player's current position. When the juggernaut collides with the player, `Game1.HandleJuggernautPlayerCollisions()` stuns the player by calling `_player.Stun(1.35f)`. The juggernaut then respawns using `juggernaut.Respawn("Stunned the player")`.

A problem with this design was the risk of stun-locking. If the juggernaut respawned too close to the player or at one fixed point, the player could be stunned repeatedly in an unfair way. To solve this, the juggernaut now uses multiple spawn points stored in `_juggernautSpawnPoints`. `Game1.GetBestJuggernautSpawnPosition()` checks which spawn points are valid and chooses the one furthest from the player. It also avoids spawn points blocked by walls, borders, or boulders. This makes the enemy more balanced and shows iteration based on testing.

8. Reflection

Overall, the project successfully combines physics simulation, AI decision-making, pathfinding, and game design iteration into one playable prototype. The strongest part of the project is how the systems interact with each other. The finite state machine gives standard thieves clear decision-making, while the pathfinder allows them to navigate a more interesting level layout. The physics system gives projectiles and boulders a meaningful gameplay role, allowing the player to use environmental hazards instead of only shooting enemies directly.

One thing that went well was the separation between AI decision-making and movement. The finite state machine decides what the thief wants to do, such as patrol, seek the relic, steal, escape, or recover from being stunned. The pathfinding and steering systems then decide how the thief moves toward that goal. This made the AI easier to modify because changing the thief's priorities did not require rewriting the whole movement system.

Another successful part was the improvement of the level design. The use of walls and corridors made the game more interesting, but it also revealed problems with simple steering because thieves could get stuck. Adding A* Pathfinding solved this and made the level design more viable. This was a useful example of how adding one feature can create a new technical problem, and how the solution can improve the overall quality of the game.

The boulder system also worked well because it added more depth to the simulation. The player can knock boulders with projectiles, and rolling boulders can defeat enemies. This creates a more dynamic game loop because the player has to think about positioning, timing, and environmental hazards.

There are still improvements that could be made in the future. The pathfinding currently uses a grid generated from wall rectangles. This works well for the current level, but a larger game could benefit from a more advanced navigation system, such as a navigation mesh or a more optimised priority queue for A*. The finite state machine could also be developed into a true hierarchical finite state machine, where high-level states such as "Stealing", "Defensive", and "Disabled" control lower-level substates. This would make the AI easier to expand if more enemy types or behaviours were added.

Another future improvement would be a level editor with serialisation. At the moment, the level layout is hard-coded in `Game1.BuildTempleRoom()`. A level editor could allow walls, spawn points, boulders, patrol paths, and relic positions to be saved to a file and loaded at runtime. This would demonstrate a stronger game development pipeline because designers could edit levels without changing the source code.

In conclusion, Temple Relic Defence meets the basic simulation requirement through its force, velocity, mass, projectile, boulder, and collision systems. It meets the basic AI requirement through a finite state machine controlling standard thief behaviour. It also goes beyond the taught material through A* pathfinding, mass-based object interactions, and an additional juggernaut enemy with adaptive respawning. These systems combine to create a more complete and interesting gameplay prototype.

Appendix A: Side Quests - Independent Study Opportunities

A.1 Introduction

This appendix presents the side quest work completed alongside the main game project. The purpose of these side quests was to extend the taught content from the module through independent study and practical experimentation. The main project already included core simulation, collision handling, steering behaviours, and AI decision making. The side quests were used to explore these areas in greater depth and improve the quality, polish, and technical ambition of the final game.

The side quest work focused on three main extensions:

1. Improving enemy navigation through pathfinding and wall avoidance.
2. Extending the physics simulation through impulse-based boulder interactions.
3. Improving the game development pipeline through reusable systems and level design iteration.

These additions were chosen because they directly supported the gameplay goals of the project. The game relies on enemies navigating through an arena, hazards interacting with different objects, and reusable systems that can be applied across more than one project. Each

side quest therefore contributed to both the final game and my wider understanding of game simulation and AI programming.

A.2 Side Quest 1: Extending AI Navigation with Pathfinding

Aim

The aim of this side quest was to extend the enemy AI beyond simple steering behaviour by implementing a pathfinding system that allowed thieves to navigate around walls, corridors, and obstacles more reliably. The original enemy behaviour worked well in open spaces, but once the level became more complex, the thieves could become stuck against walls or fail to reach the relic. This limited the level design because the arena had to remain relatively open.

Background

The module introduced AI techniques such as finite state machines and steering behaviours. These are useful for controlling what an agent wants to do and how it moves moment-to-moment. However, steering alone is not always suitable for navigating around larger obstacles. For example, an enemy can be told to seek the relic, but if a wall is between the enemy and the relic, the enemy may simply push into the wall instead of finding an alternative route. To address this, I independently explored pathfinding as an extension to the covered AI topics. Pathfinding gives an AI agent a sequence of points to follow, rather than only a single target. This allows the enemy to make more intelligent movement decisions in structured environments.

Implementation

The level was updated to include more interesting corridors, pathways, and wall layouts. This made the environment more engaging, but it also introduced new AI problems because the enemies needed to move through narrow routes without becoming trapped. To solve this, I implemented a pathfinding approach where the enemy could calculate a route around obstacles towards the relic. Instead of always moving directly towards the relic, the thief follows a path made from intermediate points. This improves navigation because the enemy is guided around blocked areas rather than repeatedly colliding with them.

The enemies also still use local movement behaviour to follow the generated path smoothly. This means the pathfinding system handles the larger navigation problem, while the movement system handles the immediate frame-by-frame motion.

Evaluation

This extension improved the game significantly because it allowed the level design to become more complex. The arena could include corridors and walls without completely breaking the

thief's behaviour. This made the gameplay feel less empty and gave the player more meaningful spaces to move through. However, there were some limitations. When pathfinding was combined with wall avoidance whisksers, the enemies could sometimes be pushed away from their intended route too strongly. This created cases where the AI appeared indecisive or became stuck near obstacles. To improve this, the wall avoidance influence had to be reduced and balanced against the path-following behaviour. Overall, this side quest showed that AI systems often need to work together carefully. A pathfinding system may solve the global navigation problem, but local steering and collision responses still need to be tuned so that they do not conflict with the path.

Reflection

This side quest helped me understand the difference between simple reactive AI and more structured navigation. Steering behaviour is effective for immediate movement, but pathfinding is more suitable when the environment contains complex obstacles. I also learned that adding a more advanced AI technique can introduce new problems when it interacts with existing systems.

In a future project, I would improve this further by adding better debug visualisation for the path nodes and enemy routes. This would make it easier to see why an enemy chooses a certain path and where it becomes stuck. I would also consider using different pathfinding settings for different enemy types, allowing faster enemies, slower enemies, or larger enemies to navigate in different ways.

A.3 Side Quest 2: Extending Physics with Boulder Impulses and Mass Differences

Aim

The aim of this side quest was to extend the physics simulation by making boulders feel heavier and more dangerous than other objects in the game. The goal was to demonstrate a clearer difference in mass between enemies and boulders. When a boulder hits an enemy, the enemy should receive a strong impulse and then die shortly after, showing that the boulder has much greater force and weight than the thief.

Background

The main project uses Newtonian physics concepts such as movement, collision, velocity, and force-like responses. However, basic collision detection only tells the game that two objects have touched. It does not automatically make the interaction feel physically meaningful. For the boulder mechanic to feel satisfying, the collision needed to communicate impact, weight, and consequence.

This side quest therefore extended the simulation aspect of the project by adding a more expressive collision response. Instead of simply detecting that a boulder touched an enemy, the game applies an impulse to the enemy and then removes or kills the enemy shortly after.

Implementation

The boulder was treated as a heavy physics object compared with the thieves. When a boulder collides with a thief, the thief receives a sudden change in velocity. This impulse pushes the thief away from the boulder, creating the visual effect of being knocked back by a much heavier object. After the impact, the thief enters a short death or removal sequence. This gives the player a clear moment to see the result of the collision before the enemy disappears. The delay is important because it makes the impact readable. If the thief disappeared instantly, the player might not clearly understand what happened. By applying impulse first, the game communicates that the boulder physically hit the thief before the enemy dies. This mechanic also supports gameplay because it allows boulders to act as environmental hazards. The player can use the boulders strategically to stop or destroy thieves, rather than only relying on direct attacks.

Evaluation

This extension improved both the simulation and the game feel. The boulders became more meaningful because they no longer behaved like ordinary objects. The difference between a thief and a boulder became visually and mechanically clear. The thief is light and can be knocked away, while the boulder is heavy and dangerous. The main challenge was balancing the impulse strength. If the impulse was too weak, the boulder did not feel powerful. If it was too strong, enemies could be launched unnaturally or behave unpredictably. The final behaviour needed to be exaggerated enough to feel satisfying while still fitting the style of the game. Another challenge was ensuring that boulders did not make the game too easy. Since boulders can kill thieves on contact, they are powerful tools. Their placement, movement, and knockback strength need to be considered carefully so that they support the gameplay rather than removing too much challenge.

Reflection

This side quest helped me better understand how physics can be used for communication, not just calculation. A technically correct collision is not always enough. The player needs to see and feel the result of the interaction. By adding impulse and delayed death behaviour, the collision became clearer and more satisfying.

In future development, I would improve this system by adding additional effects such as screen shake, sound effects, particles, or a short animation when the thief is hit. These would make the boulder impact even more readable and improve the overall polish of the game.

A.4 Side Quest 3: Improving the Development Pipeline with Reusable Systems and Level Iteration

Aim

The aim of this side quest was to improve the development process by making systems more reusable and by iterating on the level layout based on gameplay testing. The project uses a shared library for generic systems, which supports the idea of building reusable game code that can be used across multiple projects.

Background

A major focus of the module is producing reusable code and building a practical game development pipeline. Rather than writing every system directly into one game, reusable systems allow future projects to be developed more quickly and consistently. This is especially useful for common systems such as shapes, collision detection, screen management, and general utility classes. The project uses a shared game library that contains reusable building blocks. This supports a more modular structure because the main game can focus on specific gameplay logic while the shared library provides common functionality.

Implementation

The shared library was used to support general systems such as shape handling and collision checks. This allowed the main game project to use common functionality without duplicating code. For example, shape-based collision logic can be reused across multiple objects, including the player, enemies, walls, bullets, relic, and boulders.

The level also went through several iterations. The original version was more open, but this made the gameplay less interesting. Corridors, pathways, and walls were added to create a stronger arena layout. After testing, the wall density was reduced and spacing was increased so that the player and enemies had more room to move. This shows an iterative development process where the level was adjusted in response to how the game actually played. The relic size was also reduced to improve the visual balance of the scene and make the objective feel more appropriately scaled. These changes were not only visual improvements. They also affected gameplay readability, enemy movement, and player decision making.

Evaluation

This side quest improved the project in two ways. First, the reusable library helped keep the codebase more organised. Systems that are likely to be useful in other games were separated from game-specific logic. This makes the project easier to maintain and supports future development.

Second, the level iteration improved the quality of the final game. The addition of walls and corridors made the arena more interesting, while later spacing adjustments helped prevent the level from becoming too cluttered. This was important because a technically impressive level is not automatically fun to play. The layout needed to support the AI, the player movement, and the boulder mechanics at the same time. One limitation is that reusable systems require careful design. If a system is made too specific to one game, it becomes harder to reuse. If it is made too general, it can become harder to understand or slower to develop. This project helped me practise finding a balance between reusable code and practical game-specific implementation.

Reflection

This side quest helped me understand that game development is not only about implementing individual features. It is also about building a workflow that supports testing, iteration, and reuse. The level became stronger through repeated adjustment, and the shared library made the project structure cleaner. In future projects, I would continue developing reusable systems further. For example, I could create a more complete level editor, add serialisation for saving and loading level data, or build debug tools for visualising collision shapes, AI paths, and physics interactions. These additions would make the development pipeline stronger and would allow levels to be created and tested more efficiently.

A.5 Overall Conclusion

The side quest work allowed me to extend the main project beyond the basic requirements of the module. The pathfinding extension improved the AI by allowing enemies to navigate more complex environments. The boulder impulse system improved the physics simulation by making object interactions feel more meaningful and readable. The reusable systems and level iteration work improved the development pipeline and made the project easier to expand. The most important lesson from these side quests was that advanced features must be integrated carefully with existing systems. Pathfinding, steering, collision, physics, and level design all affect each other. A change that improves one area can create problems in another area, so testing and iteration are essential.

Overall, these side quests strengthened the final game and helped me develop a deeper understanding of AI, simulation, and reusable game development practices. They also provided useful experience for future projects where I will need to design systems that are both technically effective and enjoyable for players.